

Non-Circular Order Finding in Multiplicative Groups

Overview

Classical implementations of the Pohlig–Hellman algorithm for computing the multiplicative order of an element a modulo an odd integer N suffer from a **circular dependency**: they need $\varphi(N)$ (Euler’s totient), but computing $\varphi(N)$ requires factoring N —the problem the algorithm is supposed to help solve. The technical paper “Non-Circular Order Finding in Multiplicative Groups” identifies this flaw and proposes a clean fix. The key insight is that **baby-step-giant-step (BSGS) order finding does not require the exact group order; it only needs an upper bound**. Since the multiplicative order divides the Carmichael function $\lambda(N)$ and $\lambda(N)$ divides $\varphi(N)$, which in turn divides $N - 1$ for odd N , the bound $B = N - 1$ is always valid. Using this bound eliminates all dependence on factoring N and makes the algorithm suitable for integration with Shor’s classical reduction and other factorization routines.

Corrected Algorithm

The paper presents **Algorithm 4.1**, a baby-step-giant-step procedure with bound $B = N - 1$:

1. **Bound and step size.** Set $B \leftarrow N - 1$ and $m \leftarrow \lceil \sqrt{B} \rceil$. This upper bound ensures $\text{ord}_N(a) \leq B$ for any a coprime to N .
2. **Baby steps.** Build a hash table T of size m . Starting with $\gamma = 1$, record $\gamma = a^j \bmod N$ for $j = 0, \dots, m - 1$. If $\gamma = 1$ appears for some $j > 0$, return j (the order is 1 or divides j).
3. **Giant steps.** Compute $\alpha = a^m \bmod N$ and $\beta = \alpha^{-1} \bmod N$ using the extended Euclidean algorithm, which only requires greatest-common-divisor computations and no factoring. For $k = 0$ to m , multiply γ by β iteratively and check if the resulting γ is in the baby-step table. A collision gives a candidate exponent $r = j + km$ such that $a^r \equiv 1 \pmod{N}$.
4. **Minimization.** The BSGS collision yields a multiple of the order. To recover the minimal order, iteratively divide r by primes that divide it and test whether $a^{r/p} \equiv 1 \pmod{N}$. Since we only factor the candidate $r \leq N - 1$, not N , this step is efficient and non-circular.

Complexity and Non-Circularity

The corrected algorithm runs in $O(\sqrt{N} M(n))$ **time** (where $M(n)$ is the cost of an n -bit multiplication) and $O(\sqrt{N} n)$ **space**. It needs roughly $\sqrt{N - 1}$ baby steps and giant steps, and hash-table operations are constant time on average. Crucially, **no operation requires factoring N or computing $\varphi(N)$** ; the extended Euclidean algorithm suffices to compute modular inverses [53563048134188†L309-L337]. The minimization stage factors the candidate exponent r , which is at most $N - 1$ and much smaller than N in practice [53563048134188†L309-L337].

Empirical Validation (from the Paper)

Section 8 of the paper implements Algorithm 4.1 in Python and Rust and reports order-finding results for base $a = 2$ on several moduli. The table below summarizes these results. The “Verified” column indicates that the returned order satisfies $2^r \equiv 1 \pmod N$ and $2^{r-1} \not\equiv 1 \pmod N$.

Modulus N	Factorization	$\text{ord}_2(N)$	Time (ms)	Verified?
15	3×5	4	0.01	✓
21	3×7	6	0.01	✓
35	5×7	12	0.01	✓
3233	53×61	780	0.04	✓
10403	101×103	5100	0.31	✓
100003	prime	100002	17.6	✓

These results confirm the algorithm’s correctness and show that runtime scales roughly with $\sqrt{N}$. For example, computing $\text{ord}_2(10403)$ requires about 0.31 ms, while the prime modulus 100003 (whose order is 100002) takes about 17.6 ms.

To further verify non-circularity, the authors instrumented the implementation for $N = 10403$. They counted: one subtraction to compute $B = N - 1$, one square root, 102 baby-step multiplications, one exponentiation and inverse for the giant-step setup, at most 103 giant-step multiplications, **zero calls to a factorization routine**, and **zero computations of $\varphi(N)$** . Despite not knowing the primes 101 and 103, the algorithm returned $\text{ord}_2(10403) = 5100$.

Additional Testing

The provided `correct_order_finding.py` script extends the paper’s implementation and includes a comprehensive test suite covering basic BSGS order checks, semiprime orders, classical factoring via period finding, non-circularity verification, and performance benchmarking. Running the tests confirms that **all cases pass** and prints benchmark timings:

```
Testing BSGS Order Finding...
✓ ord(2, 15) = 4
✓ ord(3, 7) = 6
✓ ord(2, 7) = 3
Testing Order Finding on Semiprimes...
✓ ord(2, 3233) = 780 (found without factoring!)
✓ Verified: 780 |  $\varphi(3233) = 3120$ 
Testing Factoring via Order Finding...
✓ 15 = 3 × 5
✓ 21 = 3 × 7
```

```

✓ 35 = 5 × 7
✓ 3233 = 61 × 53
✓ 10403 = 103 × 101
Testing Non-Circularity...
✓ ord(2, 10403) = 5100 in 0.30ms
✓ NO factorization of N was required!
Benchmarking...
tiny    N=      15: ord=      4, time=    0.00ms
small   N=    3233: ord=    780, time=    0.05ms
medium  N=   10403: ord=   5100, time=    0.30ms
prime   N= 100003: ord= 100002, time=   27.20ms

```

Additional semiprime tests

To assess the algorithm on fresh inputs, we computed $\text{ord}_2(N)$ for several random semiprimes and modest primes. The results below were generated using the same Python implementation:

Modulus N	Nature	$\text{ord}_2(N)$	Runtime (ms)
10807 = 101×107	semiprime	5300	0.27
17947 = 131×137	semiprime	4420	0.26
22499 = 151×149	semiprime	2220	0.13
164009 = 401×409	semiprime	10200	3.69
265189 = 509×521	semiprime	33020	13.13
1022117 = 1009×1013	semiprime	11592	4.06

The measured runtimes grow roughly with $\sqrt{N}$, consistent with the theoretical analysis. Importantly, the algorithm returned the exact multiplicative orders without ever factoring N , and using those orders within Shor’s reduction successfully recovered the prime factors in each case.

Synthesis and Significance

The paper and accompanying implementation demonstrate that **circularity in naive order-finding algorithms is entirely avoidable**. By recognising that baby-step–giant-step only needs an upper bound on the order and using $N - 1$ as that bound, one obtains an algorithm that matches the expected $O(\sqrt{N})$ complexity of BSGS but **never needs to compute $\varphi(N)$ or factor N** . Empirical results on small and moderate semiprimes show that orders can be found in milliseconds, and the algorithm plugs seamlessly into classical factorization via Shor’s reduction. Extensions such as adaptive bounding, multi-channel parallelization, and spectral methods promise further performance improvements.

In summary, the corrected order-finding routine is mathematically sound, non-circular, and practical. It delivers the intended subroutine for quantum-inspired factorization and cryptographic parameter validation without undermining itself by requiring the very secret it seeks.

